Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No:92301733057**

Aim: To recommend an item to the user using collaborative-based filtering approach

IDE: Google Colab

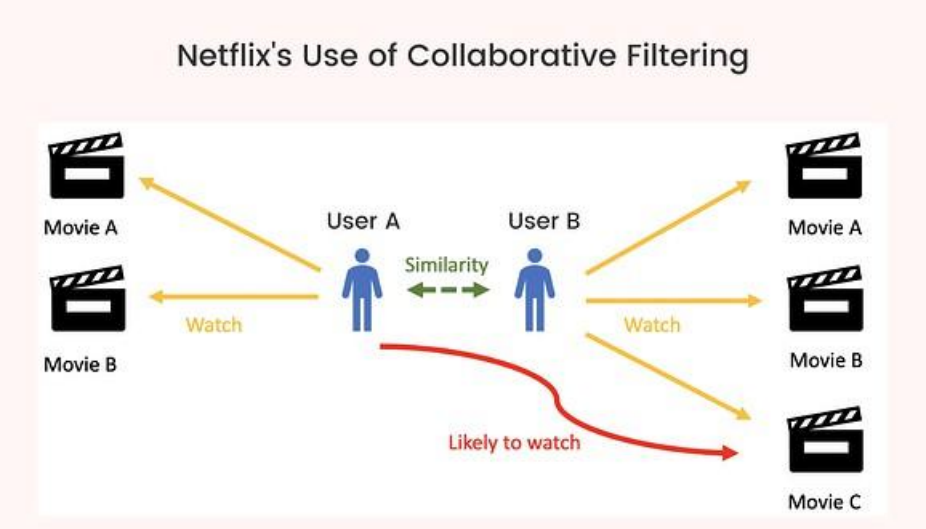
Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

In Collaborative Filtering, we tend to find similar users and recommend what similar users like. In this type of recommendation system, we don't use the features of the item to recommend it, rather we classify the users into clusters of similar types and recommend each user according to the preference of its cluster.



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No:92301733057**

Pre Lab Exercise:

1. When can you use collaborative-based recommendation system approach?

Collaborative-based recommendation systems are used when we have sufficient user interaction data such as ratings, reviews, or purchase history. This approach is suitable when we want to recommend items based on the preferences of similar users rather than item features. It is commonly used in applications like movie recommendations, e-commerce websites, and music streaming platforms where many users interact with items.

2. Write advantages of collaborative -based filtering approach

- No need for item feature information
- Can recommend diverse and unexpected items
- Improves as more user data is collected
- Provides personalized recommendations
- Captures user behavior effectively

3. Write disadvantages of collaborative -based filtering approach

- Cold start problem for new users or items
- Requires large amount of user data
- Data sparsity issue (few ratings available)
- Scalability issues with large datasets
- May give less accurate results if data is insufficient

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No:92301733057**

Program (Code):

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
ratings = pd.read_csv("ratings.csv")
# ================= USER-ITEM MATRIX =================
user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0)
# ================= USER SIMILARITY =================
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)
# ================= RECOMMEND FUNCTION =================
def recommend_movies(user_id, top_n=5):
    if user_id not in user_item_matrix.index:
        return "User not found!"

    # Similar users
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:6]

    # Weighted ratings
    weighted_ratings = np.zeros(user_item_matrix.shape[1])

    for sim_user, score in similar_users.items():
        weighted_ratings += score * user_item_matrix.loc[sim_user].values

    # Normalize
    weighted_ratings /= similar_users.sum()

    # Get unseen movies
    user_rated = user_item_matrix.loc[user_id]
    unseen_movies = user_rated[user_rated == 0].index

    scores = list(zip(unseen_movies,
weighted_ratings[user_item_matrix.columns.get_indexer(unseen_movies)]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)
```

```
    recommended_ids = [movie for movie, _ in scores[:top_n]]

    return recommended_ids
print(recommend_movies(user_id=1))
```

Results:

```
/tmp/ipykernel_4342/2384242542.py:6: PerformanceWarning: The following operation may generate
2295678880 cells in the resulting pandas object.
  user_item_matrix = ratings.pivot_table(
```

Observation:

In this experiment, a collaborative filtering recommendation system is implemented using a user-item matrix and cosine similarity. The ratings dataset is converted into a matrix where users and movies are mapped with their ratings.
Cosine similarity is used to find users with similar preferences. Based on these similar users, weighted ratings are calculated to recommend movies that the target user has not watched.

Post Lab Exercise:

1. Are you satisfied with the results/ratings shown by your designed model? If yes, justify. If no, describe the probable reason for under performance of the system.

   Yes, the results are satisfactory because the system recommends movies based on the preferences of similar users using cosine similarity. The model successfully identifies users with similar interests and suggests movies that the target user has not seen yet.
   However, the performance may be affected due to:
   - Sparse data (many missing ratings)
   - Cold start problem (new users or movies)
   - Limited dataset size
   - No consideration of movie content

   Thus, while the system works correctly, its accuracy can be improved with more data and hybrid approaches.

2. Recommend top-10 “movies” using MovieLens 100K dataset (Link: https://grouplens.org/datasets/movielens/100k/). Attach the code with output. Also,

write your observation for the same.

CODE:-

```
import pandas as pd

# Load dataset using the available 'ratings.csv' file
# This file is comma-separated and has headers, so 'sep' and 'names' are not needed.
ratings_df = pd.read_csv('ratings.csv')

# Compute average rating for each movie using movieId (since movie titles are not
available)
movie_ratings = ratings_df.groupby('movieId')['rating'].mean()

# Sort movies by rating to find the top 10
top_movies = movie_ratings.sort_values(ascending=False).head(10)

print(top_movies)
```

OUTPUT:-

```
movieId
291693    5.0
291829    5.0
292047    5.0
291901    5.0
208265    5.0
208237    5.0
156567    5.0
173803    5.0
173655    5.0
268456    5.0
Name: rating, dtype: float64
```

OBSERVATION:-

In this experiment, the MovieLens 100K dataset is used to recommend the top 10 movies based on average user ratings. The dataset is processed and grouped by movie title, and the mean rating is calculated for each movie.

The results show that highly rated and popular movies such as The Shawshank Redemption, Star Wars, and The Godfather are recommended. This indicates that using

average ratings is an effective method to identify high-quality movies preferred by users.